

ImplementMCP

May 19, 2026

1 Assignment 9: Implement a Model Context Protocol (MCP) Server

1.1 Overview

In the previous activities, you learned how to build agents with tools using `@function_tool`. The tools were defined directly in your code, and you had to manually wire them into each agent.

But what if you want your agent to connect to external services, databases, or APIs that others have already built? What if you want to share tools across multiple projects without copy/pasting code?

This is where the **Model Context Protocol (MCP)** comes in.

MCP is an open standard that allows AI agents to discover and use tools from external servers. Think of it like USB-C for AI, i.e., a universal way to plug in new capabilities. Instead of hardcoding every tool into your agent, you can connect to MCP servers that expose tools dynamically.

By the end of this assignment, you will be able to:

1. **Build an MCP server** that exposes tools for agents to use
2. **Inspect MCP servers** to discover what tools they offer
3. **Connect agents to MCP servers** using the OpenAI Agents SDK
4. **Use hosted MCP services** like gitmcp.io to access external capabilities

1.2 What is MCP?

The **Model Context Protocol (MCP)** is an open protocol that standardizes how AI applications connect to external data sources and tools. It was introduced by Anthropic in November 2024 and has since been adopted by major AI providers including OpenAI and Google.

The Three Components:

MCP uses a host-client-server architecture:

1. **MCP Host:** The AI application, like Claude Desktop, an agent framework, or IDE. It orchestrates connections and coordinates the LLM's access to tools.
2. **MCP Client:** A lightweight connector spawned by the host. Each client maintains a 1:1 connection to a single server and handles protocol messaging.
3. **MCP Server:** A service that exposes tools, resources, and context. Servers advertise their capabilities, and clients discover them at runtime.

When you use the OpenAI Agents SDK with `mcp_servers=[server]`, the SDK acts as the **host** and creates **clients** to connect to your **servers**.

The Key Insight:

You can build a server once, and any MCP-compatible host can connect to it. This enables:

- **Reusability:** Write a tool once, use it from many applications
- **Ecosystem:** Use tools built by others (databases, APIs, services)
- **Dynamic discovery:** Hosts discover what tools exist at runtime

1.2.1 Initial Setup

Let's import the libraries we'll need:

- **Agent, Runner:** The core classes for creating and running agents (you've used these before)
- **MCPServerStdio:** Connects to local MCP servers running as subprocesses
- **HostedMCPTool:** Connects to remote/hosted MCP services

Run the Following Cell

```
[1]: # === Imports ===
import asyncio
import uuid
import json
import openai_setup
from IPython.display import display, Markdown
import os
os.environ["IPYTHON_HISTORY_FILE"] = ":memory:"

# === Agents SDK ===
from agents import Agent, Runner, set_tracing_disabled
from agents.mcp import MCPServerStdio # For connecting to local MCP servers
from agents import HostedMCPTool     # For connecting to hosted MCP services

set_tracing_disabled(True)
```

2 Part 1: Sticky Notes MCP Server

Let's learn MCP by building, inspecting, and using a complete example: a **sticky notes** server.

You will see the full workflow: 1. **Build** the server with `@mcp.tool()` decorators 2. **Inspect** it to discover available tools 3. **Use** it with an agent to create and search notes

2.1 What is an MCP Server?

An MCP server is a standalone program, often a Python script, that exposes tools via the Model Context Protocol. When an agent connects to the server, it can automatically discover which tools are available, along with their names, descriptions, and input schemas.

MCP servers run as **separate processes** from your agent. This design provides: - **Isolation**: The server runs independently, so crashes don't affect your agent - **Reusability**: Multiple agents, or applications, can connect to the same server - **Language flexibility**: Servers can be written in any language, as long as they implement MCP

Because MCP servers are separate programs, we'll implement our MCP servers as standalone Python .py programs and run them as subprocesses.

What is FastMCP?

FastMCP is a Python library that makes it easy to build MCP servers. It handles all the protocol details for you. You just have to do the following: 1. Create a `FastMCP` instance with a name 2. Decorate your functions with `@mcp.tool()` 3. Call `mcp.run()` to start the server

FastMCP automatically: - Generates tool schemas from your function signatures and docstrings - Handles the MCP protocol communication - Manages connections from clients

The Basic Structure:

```
from mcp.server.fastmcp import FastMCP

# 1. Initialize the server
mcp = FastMCP("my-server-name")

# 2. Define tools with the decorator
@mcp.tool()
def my_tool(param: str) -> dict:
    """Tool description for the agent."""
    return {"result": param}

# 3. Start the server
if __name__ == "__main__":
    mcp.run()
```

The `@mcp.tool()` decorator is similar to `@function_tool` from the Agents SDK, but instead of defining tools inline with your agent, it exposes them via the MCP protocol so any compatible client can use them.

2.2 Example: Sticky Notes MCP Server

Let's build a **sticky notes** server that lets agents create, search, and delete notes.

Why Write to a File?

Since MCP servers run as separate processes, we need to save our server code to a standalone .py file. When the agent connects, the Agents SDK launches this file as a subprocess and communicates with it using the MCP protocol.

For local MCP servers, the agent and server typically communicate over standard input and standard output (`stdio`). The agent sends structured requests through `stdin`, and the server returns structured responses through `stdout`.

The code cell below: 1. Defines the server code as a Python string 2. Writes it to `sticky_notes_server.py`

Later, we'll connect to this server using `MCPServerStdio`.

This pattern of writing server code to files is common when building MCP servers in Jupyter notebooks.

What This Server Does:

- **In-memory storage:** Uses a simple Python dictionary to store notes
- **Three tools:** add, search, delete
- **Logging:** Prints [MCP] messages so you can see when tools are called

Run the Following Cell

```
[2]: # === Write the Sticky Notes MCP Server to a File ===

server_code = '''
import uuid
from mcp.server.fastmcp import FastMCP

# Initialize MCP server
mcp = FastMCP("sticky-notes")

# In-memory store
NOTES = {}

def search_notes(query: str):
    """Simple substring search across titles and bodies."""
    results = []
    for note_id, note in NOTES.items():
        if query.lower() in note["title"].lower() or query.lower() in_
↪note["body"].lower():
            snippet = note["body"][:50] + ("..." if len(note["body"]) > 50 else_
↪...)
            results.append({"id": note_id, "title": note["title"], "snippet":_
↪snippet})
    return results

# -----
# TOOLS
# -----
@mcp.tool()
def add(title: str, body: str) -> dict:
    """
    Add a sticky note.

    Args:
        title: Title of the note

```

```

        body: Body text of the note

Returns:
    dict: { id }
    """
    note_id = str(uuid.uuid4())
    NOTES[note_id] = {"title": title, "body": body}
    print(f"[MCP] Added note: {title}")
    return {"id": note_id}

@mcp.tool()
def search(query: str) -> dict:
    """
    Search sticky notes by query substring.

    Args:
        query: Substring to look for in title or body

    Returns:
        dict: { results: [{ id, title, snippet }] }
    """
    print(f"[MCP] Searching for: {query}")
    return {"results": search_notes(query)}

@mcp.tool()
def delete(id: str) -> dict:
    """
    Delete a sticky note.

    Args:
        id: Note ID to delete

    Returns:
        dict: { ok: bool }
    """
    if id in NOTES:
        del NOTES[id]
        print(f"[MCP] Deleted note: {id}")
        return {"ok": True}
    return {"ok": False}

if __name__ == "__main__":
    mcp.run()

```

Write to file

```

with open("sticky_notes_server.py", "w") as f:

```

```
f.write(server_code)

print("Created: sticky_notes_server.py")
print("\nThis MCP server provides 3 tools: add, search, delete")
```

Created: sticky_notes_server.py

This MCP server provides 3 tools: add, search, delete

2.2.1 Anatomy of the Server

Let's break down what we just created:

Component	Purpose
FastMCP("sticky-notes")	Creates the server with a name
NOTES = {}	In-memory storage (a simple dictionary)
@mcp.tool()	Decorator that exposes a function as a tool
mcp.run()	Starts the server and listens for connections

How Tool Discovery Works:

The `@mcp.tool()` decorator automatically generates a schema from your function's docstring and type hints. This schema tells agents: - What the tool does (from the docstring) - What parameters it accepts (from type hints) - What it returns

When an agent connects to the server, it receives this schema and can decide when and how to use each tool.

2.3 Inspecting MCP Servers

Before using an MCP server, you can inspect it to see what tools it offers. This is how agents discover capabilities at runtime.

What is MCPServerStdio?

`MCPServerStdio` is a class from the Agents SDK that connects to local MCP servers. The name comes from how it communicates: via **Standard Input/Output (STDIO)**.

When you create an `MCPServerStdio` connection: 1. The SDK launches your server script as a subprocess 2. It communicates with the server through `stdin/stdout` pipes 3. You can then call methods like `list_tools()` to discover available tools

The Connection Pattern:

```
server = MCPServerStdio(
    params={"command": "python", "args": ["my_server.py"]}
)

async with server:
    tools = await server.list_tools()  # Discover available tools
```

The `async with server:` block ensures the connection is properly opened and closed.

Run the Following Cell

```
[3]: # === Inspect the Sticky Notes Server ===

async def inspect_mcp_server():
    """Connect to an MCP server and list its tools with full details."""

    # Create connection to our server
    server = MCPServerStdio(
        params={"command": "python", "args": ["sticky_notes_server.py"]},
        client_session_timeout_seconds=60 # increase timeout from default 5
    )

    # Connect and discover tools
    async with server:
        tools = await server.list_tools()

        print("=" * 60)
        print("STICKY NOTES MCP SERVER - Tool Inspection")
        print("=" * 60)
        print(f"\nDiscovered {len(tools)} tools:\n")

        for tool in tools:
            print(f"{'='*60}")
            print(f"TOOL: {tool.name}")
            print(f"{'='*60}")
            print(f>Description: {tool.description}")
            print(f"\nParameters:")

            # Show the input schema if available
            if hasattr(tool, 'inputSchema') and tool.inputSchema:
                schema = tool.inputSchema
                if 'properties' in schema:
                    for param_name, param_info in schema['properties'].items():
                        param_type = param_info.get('type', 'unknown')
                        param_desc = param_info.get('description', 'No
↪description')

                        required = param_name in schema.get('required', [])
                        req_marker = " (required)" if required else "
↪(optional)"

                        print(f"  - {param_name}: {param_type}{req_marker}")
                        print(f"    {param_desc}")
                else:
                    print("  No parameters")
            else:
                print("  Schema not available")
```

```
print()

await inspect_mcp_server()
```

```
=====
STICKY NOTES MCP SERVER - Tool Inspection
=====
```

Discovered 3 tools:

```
=====
TOOL: add
=====
```

Description:
Add a sticky note.

Args:
 title: Title of the note
 body: Body text of the note

Returns:
 dict: { id }

Parameters:
 - title: string (required)
 No description
 - body: string (required)
 No description

```
=====
TOOL: search
=====
```

Description:
Search sticky notes by query substring.

Args:
 query: Substring to look for in title or body

Returns:
 dict: { results: [{ id, title, snippet }] }

Parameters:
 - query: string (required)
 No description


```

=====
TOOL: delete
=====
Description:
Delete a sticky note.

Args:
    id: Note ID to delete

Returns:
    dict: { ok: bool }

Parameters:
    - id: string (required)
      No description

```

2.3.1 What Just Happened?

1. We created an `MCPServerStdio` connection pointing to our server script
2. The SDK launched the server as a subprocess via `STDIO` transport
3. We called `list_tools()` to discover available tools
4. The server reported back its three tools: `add`, `search`, `delete`

This is the **discovery** phase. When an agent connects to an MCP server, it performs this same discovery to learn what tools are available. The agent doesn't need to know in advance what tools exist. It discovers them dynamically.

The `MCPServerStdio` Connection:

```

server = MCPServerStdio(
    params={"command": "python", "args": ["sticky_notes_server.py"]}
)

```

This tells the SDK: - **command**: The program to run (`python`) - **args**: Arguments to pass (`["sticky_notes_server.py"]`)

The SDK launches this as a subprocess and communicates via standard input/output (`STDIO`).

2.4 Using the Sticky Notes Server With an Agent

Now let's connect an agent to the sticky notes server and see it in action.

The Connection Pattern:

1. Create an `MCPServerStdio` connection pointing to your server file
2. Use `async with server:` to start the server subprocess
3. Pass the server to the agent via `mcp_servers=[server]`
4. The agent automatically discovers and can use all the server's tools

Understanding the Code:

In the cell below, we also define two helper functions for tracing tool calls: - `_as_dict()`: Converts SDK objects to dictionaries for inspection - `print_tool_trace()`: Shows the back-and-forth between model and tools

These helpers let you see exactly what MCP calls are being made.

Run the Following Cell

```
[4]: # === Use the Sticky Notes Server with an Agent ===
from agents.items import ToolCallItem, ToolCallOutputItem

def _as_dict(obj):
    """Convert SDK objects to dictionaries for inspection."""
    if isinstance(obj, dict):
        return obj
    if hasattr(obj, "model_dump"):
        return obj.model_dump(exclude_unset=True)
    return {"repr": repr(obj)}

def print_tool_trace(result, label=""):
    """Print tool call trace from a runner result."""
    print(f"\n--- Tool Trace{' (' + label + ' ' if label else ''} ---")
    for item in result.new_items:
        if isinstance(item, ToolCallItem):
            d = _as_dict(item.raw_item)
            print(f"MODEL -> TOOL: {d.get('name')} args={d.get('arguments')}")
        elif isinstance(item, ToolCallOutputItem):
            print(f"TOOL -> MODEL: {item.output}")

async def demo_sticky_notes():
    # Connect to our MCP server
    server = MCPServerStdio(
        params={"command": "python", "args": ["sticky_notes_server.py"]}
    )

    async with server:
        # Create an agent with access to the MCP server
        agent = Agent(
            name="Notes Assistant",
            instructions="You help users manage their sticky notes. Use the
↪available tools.",
            model="gpt-4.1",
            mcp_servers=[server] # <-- This connects the agent to the MCP
↪server
        )

        # Ask the agent to create a note
        print("User: Create a note about buying groceries")
```

```

print("=" * 60)

result = await Runner.run(agent, "Create a note titled 'Shopping' about_
↳buying milk, eggs, and bread")

print(f"\nAgent: {result.final_output}")
print_tool_trace(result)

await demo_sticky_notes()

```

User: Create a note about buying groceries

=====

Agent: The note titled "Shopping" about buying milk, eggs, and bread has been created. If you need to add more items or make changes, let me know!

--- Tool Trace ---

MODEL -> TOOL: add args={"title":"Shopping","body":"Buy milk, eggs, and bread."}

TOOL -> MODEL: {"type":"text","text":{"\n \n"id\":"9f3952e7-e319-41ce-a149-135d25826a66"\n"},"annotations":null,"meta":null}

2.4.1 What Just Happened?

Look at the **Tool Trace** output above. This shows the MCP interaction:

1. **MODEL** → **TOOL**: The agent decided to call `add` with the title and body
2. **TOOL** → **MODEL**: The server returned a response with the new note's ID

The agent didn't need any special configuration for the sticky notes tools. It discovered them from the MCP server and used them appropriately based on the user's request.

2.5 Multi-Turn Interactions With MCP

MCP servers maintain state between calls. Let's see this with a more complex interaction that creates multiple notes and then searches for them.

Run the Following Cell

[5]: *# === Multi-Turn Interaction with Sticky Notes ===*

```

async def demo_multi_turn_sticky():
    server = MCPServerStdio(
        params={"command": "python", "args": ["sticky_notes_server.py"]},
        client_session_timeout_seconds=60  # increase timeout from default 5
    )

    async with server:
        agent = Agent(

```

```

        name="Notes Assistant",
        instructions="You help users manage their sticky notes. Use the
↳available tools.",
        model="gpt-4.1",
        mcp_servers=[server]
    )

    # Turn 1: Create first note
    print("Turn 1: Creating first note...")
    print("=" * 60)
    result1 = await Runner.run(agent, "Create a note titled 'Meeting' about
↳the 3pm project review")
    print(f"Agent: {result1.final_output}")
    print_tool_trace(result1, "Turn 1")

    # Turn 2: Create second note
    print("\n" + "=" * 60)
    print("Turn 2: Creating second note...")
    print("=" * 60)
    result2 = await Runner.run(agent, "Create a note titled 'Ideas' about
↳machine learning project ideas")
    print(f"Agent: {result2.final_output}")
    print_tool_trace(result2, "Turn 2")

    # Turn 3: Search for notes
    print("\n" + "=" * 60)
    print("Turn 3: Searching for notes...")
    print("=" * 60)
    result3 = await Runner.run(agent, "Search for notes about 'project'")
    print(f"Agent: {result3.final_output}")
    print_tool_trace(result3, "Turn 3")

    await demo_multi_turn_sticky()

```

Turn 1: Creating first note..

=====

Agent: A note titled 'Meeting' about the 3pm project review has been created. If you need to add more details or create another note, let me know!

--- Tool Trace (Turn 1) ---

MODEL -> TOOL: add args={"title":"Meeting","body":"3pm project review"}

TOOL -> MODEL: {"type":"text","text":{"\n \n id\n":
"\n9f5491aa-2c5b-4b4a-a937-58738fdb4e53\n\n"},"annotations":null,"meta":null}

=====

Turn 2: Creating second note..

=====

Agent: A note titled "Ideas" about machine learning project ideas has been created. If you want to add specific project ideas to this note or need more details, let me know!

```
=====
Turn 3: Searching for notes...
```

1. Title: Meeting
- Snippet: "3pm project review"
2. Title: Ideas
- Snippet: "Machine learning project ideas"

```

--- Tool Trace (Turn 3) ---
MODEL -> TOOL: search  args={"query":"project"}
TOOL -> MODEL: {"type":"text","text":{"\n  \"results\": [\n    {\n      \"id\":  
\"9f5491aa-2c5b-4b4a-a937-58738fdb4e53\", \n      \"title\": \"Meeting\", \n      \"snippet\": \"3pm project review\" \n    }, \n    {\n      \"id\":  
\"cd94cd07-7356-4912-9634-01fa591c5d7c\", \n      \"title\": \"Ideas\", \n      \"snippet\": \"Machine learning project ideas\" \n    } \n  ] \n}, \"annotations\":null, \"meta\":null}

```

Notice that the search found both notes we created. This is because:

This is a key advantage of MCP: The server can maintain persistent state that multiple agent calls can access.

3 Part 2: Todo List MCP Server

Now it's your turn. You'll work with a **todo list** MCP server and connect it to an agent.

The workflow follows the same pattern you just saw: 1. **Build** the server (provided for you) 2. **Inspect** it to see the available tools 3. **Use** it with an agent (this is the graded part)

3.1 Building the Todo Server

The todo server has three tools: - **add_task**: Add a new task to the list - **list_tasks**: Show all tasks with their status - **complete_task**: Mark a task as done

Study the code below. It follows the same patterns as the sticky notes server.

Run the Following Cell

```
[6]: # === Build a Todo List MCP Server ===

todo_server_code = '''
import uuid
from mcp.server.fastmcp import FastMCP

# Initialize MCP server
mcp = FastMCP("todo-list")

# In-memory store: {id: {"task": str, "done": bool}}
TASKS = {}

@mcp.tool()
def add_task(task: str) -> dict:
    """
    Add a new task to the todo list.

    Args:
        task: Description of the task

    Returns:
        dict: { id, task, done }
    """
    task_id = str(uuid.uuid4())
    TASKS[task_id] = {"task": task, "done": False}
    print(f"[MCP] Added task: {task}")
    return {"id": task_id, "task": task, "done": False}

@mcp.tool()
def list_tasks() -> dict:
    """
    List all tasks in the todo list.
```

```

Returns:
    dict: { tasks: [{ id, task, done }] }
    """
    print(f"[MCP] Listing {len(TASKS)} tasks")
    tasks = [{"id": tid, **data} for tid, data in TASKS.items()]
    return {"tasks": tasks}

@mcp.tool()
def complete_task(id: str) -> dict:
    """
    Mark a task as completed.

    Args:
        id: Task ID to mark as done

    Returns:
        dict: { ok: bool, task: str }
        """
    if id in TASKS:
        TASKS[id]["done"] = True
        print(f"[MCP] Completed task: {TASKS[id]['task']}")
        return {"ok": True, "task": TASKS[id]["task"]}
    return {"ok": False, "task": None}

if __name__ == "__main__":
    mcp.run()
    ...

# Write to file
with open("todo_server.py", "w") as f:
    f.write(todo_server_code)

print("Created: todo_server.py")
print("\nThis MCP server provides 3 tools: add_task, list_tasks, complete_task")

```

Created: todo_server.py

This MCP server provides 3 tools: add_task, list_tasks, complete_task

3.2 Inspecting the Todo Server

Let's inspect the todo server to see its tools.

Run the Following Cell

```

[7]: # === Inspect Your Todo Server ===

async def inspect_todo_server():

```

```

server = MCPServerStdio(
    params={"command": "python", "args": ["todo_server.py"]},
    client_session_timeout_seconds=60  # increase timeout from default 5
)

async with server:
    tools = await server.list_tools()

    print("=" * 60)
    print("TODO LIST MCP SERVER - Tool Inspection")
    print("=" * 60)
    print(f"\nDiscovered {len(tools)} tools:\n")

    for tool in tools:
        print(f"{'='*60}")
        print(f"TOOL: {tool.name}")
        print(f"{'='*60}")
        print(f>Description: {tool.description}")
        print(f"\nParameters:")

        # Show the input schema if available
        if hasattr(tool, 'inputSchema') and tool.inputSchema:
            schema = tool.inputSchema
            if 'properties' in schema:
                for param_name, param_info in schema['properties'].items():
                    param_type = param_info.get('type', 'unknown')
                    param_desc = param_info.get('description', 'No
↳description')

                    required = param_name in schema.get('required', [])
                    req_marker = " (required)" if required else "
↳(optional)"

                    print(f"  - {param_name}: {param_type}{req_marker}")
                    print(f"    {param_desc}")
            else:
                print("    No parameters")
        else:
            print("    Schema not available")
        print()

    await inspect_todo_server()

```

```

=====
TODO LIST MCP SERVER - Tool Inspection
=====

```

Discovered 3 tools:


```

=====
TOOL: add_task
=====
Description:
Add a new task to the todo list.

Args:
    task: Description of the task

Returns:
    dict: { id, task, done }

Parameters:
    - task: string (required)
      No description

=====
TOOL: list_tasks
=====
Description:
List all tasks in the todo list.

Returns:
    dict: { tasks: [{ id, task, done }] }

Parameters:

=====
TOOL: complete_task
=====
Description:
Mark a task as completed.

Args:
    id: Task ID to mark as done

Returns:
    dict: { ok: bool, task: str }

Parameters:
    - id: string (required)
      No description

```

3.3 Graded Cell: Use the Todo Server in a Multi-Turn Interaction

The code cell below sets up the MCP server connection and the multi-turn interaction sequence. Although the server has already been created using `MCPServerStdio`, it runs as a separate process from the agent. Because of this separation, the agent does not automatically know about the tools exposed by the server. The server must be explicitly passed to the agent to allow it to discover and invoke the available todo tools during the interaction.

Your task is to complete the `Agent` definition by adding the `mcp_servers` parameter so the agent can discover and use the todo tools.

Hint: Look at the sticky notes examples above to see the pattern it uses.

After running the cell, you should see: 1. The agent adding multiple tasks 2. The agent listing tasks 3. The agent marking a task as complete 4. Tool call traces showing each MCP tool invocation

3.3.1 Graded Cell

The cell below will be graded. Replace the lines `raise NotImplementedError("Your code is missing.")` with your code. Enter your solution and then run the following cell.

```
[8]: # === Connect Your Todo Server to an Agent (Multi-Turn) ===

async def demo_todo_multi_turn():
    # Connect to your todo MCP server
    server = MCPServerStdio(
        params={"command": "python", "args": ["todo_server.py"]},
        client_session_timeout_seconds=60 # increase timeout from default 5
    )

    async with server:
        # Create an agent with access to the MCP server
        # Complete the mcp_servers parameter to connect the agent
        agent = Agent(
            name="Todo Assistant",
            instructions="You help users manage their todo list. Use the_
↪available tools to add, list, and complete tasks.",
            model="gpt-4.1",
            ### BEGIN SOLUTION
            mcp_servers=[server]
            ### END SOLUTION
        )

        # Turn 1: Add some tasks
        print("Turn 1: Adding tasks...")
        print("=" * 60)
        result1 = await Runner.run(agent, "Add these tasks to my todo list: buy_
↪groceries, call mom, finish report")
        print(f"Agent: {result1.final_output}")
        print_tool_trace(result1, "Turn 1")
```

```

# Turn 2: List all tasks
print("\n" + "=" * 60)
print("Turn 2: Listing tasks...")
print("=" * 60)
result2 = await Runner.run(agent, "Show me all my tasks")
print(f"Agent: {result2.final_output}")
print_tool_trace(result2, "Turn 2")

# Turn 3: Complete a task
print("\n" + "=" * 60)
print("Turn 3: Completing a task...")
print("=" * 60)
result3 = await Runner.run(agent, "I finished calling mom, mark that_
↳task as complete")
print(f"Agent: {result3.final_output}")
print_tool_trace(result3, "Turn 3")

# Turn 4: List remaining tasks
print("\n" + "=" * 60)
print("Turn 4: Listing remaining tasks...")
print("=" * 60)
result4 = await Runner.run(agent, "What tasks do I still have left to_
↳do?")
print(f"Agent: {result4.final_output}")
print_tool_trace(result4, "Turn 4")

await demo_todo_multi_turn()

```

Turn 1: Adding tasks...

=====

Agent: All three tasks have been added to your todo list:

1. Buy groceries
2. Call mom
3. Finish report

Would you like to view your full todo list or manage any tasks now?

--- Tool Trace (Turn 1) ---

MODEL -> TOOL: add_task args={"task":"buy groceries"}

MODEL -> TOOL: add_task args={"task":"call mom"}

MODEL -> TOOL: add_task args={"task":"finish report"}

TOOL -> MODEL: {"type":"text","text":{"\n \n "id":
"\n9f72dbed-4114-4be4-9736-6ebd8fc78b3e\n",\n \n "task": "\nbuy groceries\n",\n
"\ndone": false\n"},"annotations":null,"meta":null}

TOOL -> MODEL: {"type":"text","text":{"\n \n "id":
"\n23a399f0-61a8-4e2b-9d22-661ca71039de\n",\n \n "task": "\ncall mom\n",\n

```
\\"done\\": false\\n\\", "annotations": null, "meta": null}
TOOL -> MODEL: {"type": "text", "text": "{\\n  \\\"id\\\":
\\\"4f210b03-e8ce-4615-92ab-459dd66b15a6\\\",\\n  \\\"task\\\": \\\"finish report\\\",\\n
\\\"done\\\": false\\n\\", "annotations": null, "meta": null}
```

=====
Turn 2: Listing tasks..
=====

Agent: Here are all your current tasks:

1. Buy groceries
2. Call mom
3. Finish report

Let me know if you want to add, complete, or edit any tasks!

--- Tool Trace (Turn 2) ---

```
MODEL -> TOOL: list_tasks args={}
TOOL -> MODEL: {"type": "text", "text": "{\\n  \\\"tasks\\\": [\\n    {\\n      \\\"id\\\":
\\\"9f72dbed-4114-4be4-9736-6ebd8fc78b3e\\\",\\n      \\\"task\\\": \\\"buy groceries\\\",\\n
\\\"done\\\": false\\n    },\\n    {\\n      \\\"id\\\":
\\\"23a399f0-61a8-4e2b-9d22-661ca71039de\\\",\\n      \\\"task\\\": \\\"call mom\\\",\\n
\\\"done\\\": false\\n    },\\n    {\\n      \\\"id\\\":
\\\"4f210b03-e8ce-4615-92ab-459dd66b15a6\\\",\\n      \\\"task\\\": \\\"finish report\\\",\\n
\\\"done\\\": false\\n    }\\n  ]\\n\\", "annotations": null, "meta": null}
```

=====
Turn 3: Completing a task..
=====

Agent: The task "call mom" has been marked as complete. If you need help with anything else on your todo list, just let me know!

--- Tool Trace (Turn 3) ---

```
MODEL -> TOOL: list_tasks args={}
TOOL -> MODEL: {"type": "text", "text": "{\\n  \\\"tasks\\\": [\\n    {\\n      \\\"id\\\":
\\\"9f72dbed-4114-4be4-9736-6ebd8fc78b3e\\\",\\n      \\\"task\\\": \\\"buy groceries\\\",\\n
\\\"done\\\": false\\n    },\\n    {\\n      \\\"id\\\":
\\\"23a399f0-61a8-4e2b-9d22-661ca71039de\\\",\\n      \\\"task\\\": \\\"call mom\\\",\\n
\\\"done\\\": false\\n    },\\n    {\\n      \\\"id\\\":
\\\"4f210b03-e8ce-4615-92ab-459dd66b15a6\\\",\\n      \\\"task\\\": \\\"finish report\\\",\\n
\\\"done\\\": false\\n    }\\n  ]\\n\\", "annotations": null, "meta": null}
MODEL -> TOOL: complete_task args={"id": "23a399f0-61a8-4e2b-9d22-661ca71039de"}
TOOL -> MODEL: {"type": "text", "text": "{\\n  \\\"ok\\\": true,\\n  \\\"task\\\": \\\"call
mom\\\"\\n\\", "annotations": null, "meta": null}
```

=====
Turn 4: Listing remaining tasks..
=====

Agent: You still have the following tasks left to do:

1. Buy groceries
2. Finish report

Would you like to mark any of these as completed or add new tasks?

--- Tool Trace (Turn 4) ---

MODEL -> TOOL: list_tasks args={}

```
TOOL -> MODEL: {"type": "text", "text": "{\n  \"tasks\": [\n    {\n      \"id\":\n        \"9f72dbed-4114-4be4-9736-6ebd8fc78b3e\",\n      \"task\": \"buy groceries\",\n      \"done\": false\n    },\n    {\n      \"id\":\n        \"23a399f0-61a8-4e2b-9d22-661ca71039de\",\n      \"task\": \"call mom\",\n      \"done\": true\n    },\n    {\n      \"id\":\n        \"4f210b03-e8ce-4615-92ab-459dd66b15a6\",\n      \"task\": \"finish report\",\n      \"done\": false\n    }\n  ]\n}", "annotations": null, "meta": null}
```

4 Part 3: Hosted MCP Services

You don't always need to build your own MCP server. There are hosted services that provide MCP access to external capabilities.

One popular example is **gitmcp.io**, a hosted MCP service that provides an MCP interface to GitHub repositories. This lets agents read and explore codebases without you writing any server code.

Given a repository URL, **gitmcp.io** provides tools to:

- Read files from the repository
- Search for code patterns
- Understand the project structure

The URL Pattern:

`https://gitmcp.io/{owner}/{repo}`

For example: - `https://gitmcp.io/openai/codex`: OpenAI's Codex repository -
`https://gitmcp.io/anthropics/claude-cookbooks`: Anthropic's cookbook repository

What is HostedMCPTool?

HostedMCPTool is a class from the Agents SDK for connecting to **remote/hosted** MCP services. Unlike **MCPServerStdio** which runs a local subprocess, **HostedMCPTool** connects to services running on external infrastructure.

The key difference is where you pass it: - Local servers: `Agent(mcp_servers=[server])` - Hosted services: `Agent(tools=[HostedMCPTool(...)])`

The tool_config Dictionary:

When creating a **HostedMCPTool**, you provide a **tool_config** dictionary with: - **type**: Always "mcp" for MCP services - **server_label**: A name for the service (for logging/debugging) -

server_url: The URL of the hosted MCP service - require_approval: Whether to ask before using tools ("never", "always")

```
agent = Agent(
    tools=[
        HostedMCPTool(
            tool_config={
                "type": "mcp",
                "server_label": "gitmcp",
                "server_url": "https://gitmcp.io/openai/codex",
                "require_approval": "never",
            }
        )
    ]
)
```

4.0.1 Demo: Exploring a Repository With gitmcp.io

Let's use gitmcp.io to explore OpenAI's Codex repository.

Run the Following Cell

```
[9]: # === Explore a Repository with gitmcp.io ===

async def explore_repo():
    # Create an agent with access to the OpenAI Codex repo via gitmcp.io
    agent = Agent(
        name="Code Explorer",
        instructions="You help users understand codebases. Use the available_
↳ tools to read and explore code. Be concise.",
        model="gpt-4.1",
        tools=[
            HostedMCPTool(
                tool_config={
                    "type": "mcp",
                    "server_label": "gitmcp",
                    "server_url": "https://gitmcp.io/openai/codex",
                    "require_approval": "never",
                }
            )
        ]
    )

    print("User: What is this repository about?")
    print("=" * 50)

    result = await Runner.run(
        agent,
```

```

        "What is this repository? Give me a one-paragraph summary based on the
        ↪README."
    )

    print(f"\nAgent:\n{result.final_output}")

await explore_repo()

```

User: What is this repository about?

=====

Agent:

The openai/codex repository contains the source code, documentation, and configuration for the OpenAI Codex CLI, a command-line tool powered by OpenAI Codex models. Designed for developers, it offers features like interactive and non-interactive modes, agent configuration, managed lifecycle hooks, and execution policies to streamline code explanation and automation tasks. The repository includes instructions for installation and building (primarily in Rust), guides for configuration and authentication, information on sandboxing and security, as well as references for advanced features such as slash commands and skills. Additionally, the project emphasizes openness, with an Apache-2.0 license and an open source fund supporting external projects built on Codex CLI.

4.0.2 What Just Happened?

Notice how we connected to a GitHub repository without: - Writing any server code - Managing GitHub API tokens - Handling rate limits or pagination

The hosted MCP service handles all of that. We just provide the URL in `tool_config` and the agent gets access to tools for exploring the repository.

Key Differences: Local vs. Hosted

Aspect	Local (MCPServerStdio)	Hosted (HostedMCPTool)
Parameter	<code>mcp_servers=[server]</code>	<code>tools=[HostedMCPTool(tool_config={...})]</code>
Server	You run it locally	Someone else hosts it
Setup	Write server code	Just provide URL in config
State	Persists in your process	Managed by service
Execution	Runs in subprocess	Runs on remote infrastructure

4.1 Graded Cell: Use a Hosted MCP Service

Now it's your turn to connect an agent to a hosted MCP service.

The code cell below configures a hosted MCP tool (`HostedMCPTool`) that connects to a remote MCP service backed by a GitHub repository. It then defines an agent that uses this tool to explore the repository and answer a question about it.

Unlike the previous example, which launched a local MCP server as a subprocess, this example connects to a hosted MCP service. Because the agent does not automatically have access to hosted tools, you must explicitly pass the tool via the `tools` parameter.

Your task is to complete the `Agent` definition by adding the `tools` parameter so the agent can query the repository through the hosted MCP service

Hint: Look at the sticky notes examples above to see the pattern it uses.

4.1.1 Graded Cell

The cell below will be graded. Replace the line `raise NotImplementedError("Your code is missing.")` with your code. Enter your solution and then run the following cell.

```
[10]: # === Use a Hosted MCP Service ===

# The HostedMCPTool is already configured for you
hosted_tool = HostedMCPTool(
    tool_config={
        "type": "mcp",
        "server_label": "gitmcp",
        "server_url": "https://gitmcp.io/anthropics/claude-cookbooks",
        "require_approval": "never",
    }
)

async def explore_with_hosted_mcp():
    # Create an agent and connect it to the hosted MCP service
    # Complete the tools parameter to give the agent access
    agent = Agent(
        name="Code Explorer",
        instructions="You help users understand codebases. Be concise and focus_
↪on key features.",
        model="gpt-4.1",
        ### BEGIN SOLUTION
        tools=[hosted_tool]
        ### END SOLUTION
    )

    print("User: What is this repository about?")
    print("=" * 60)

    result = await Runner.run(
        agent,
        "What is this repository? Summarize the main purpose in 2-3 sentences_
↪based on the README."
    )

    print(f"\nAgent:\n{result.final_output}")
```



```
await explore_with_hosted_mcp()
```

User: What is this repository about?

=====

Agent:

This repository, "Claude Cookbooks," provides a collection of code examples and guides to help developers build applications using the Claude API. It covers practical recipes for tasks such as text classification, summarization, retrieval augmented generation, tool integration, vision capabilities, and third-party integrations. The cookbooks are primarily in Python and are designed for easy copy-and-paste integration into real projects.

4.2 Summary

In this project, you learned how to build and use MCP servers with AI agents.

4.2.1 Key Concepts

Concept	What You Learned
Building MCP Servers	Use FastMCP and <code>@mcp.tool()</code> to expose functions as tools
Inspecting Servers	Use <code>list_tools()</code> to discover available tools at runtime
Local Connections	Use <code>MCPServerStdio + mcp_servers=[server]</code> for local servers
Hosted Services	Use <code>HostedMCPTool(tool_config={...})</code> for hosted MCP services

4.2.2 The MCP Server Pattern

```
from mcp.server.fastmcp import FastMCP
```

```
mcp = FastMCP("server-name")
```

```
@mcp.tool()
def my_tool(param: str) -> dict:
    """Tool description."""
    return {"result": param}

if __name__ == "__main__":
    mcp.run()
```

4.2.3 Connecting Agents to MCP

```
# Local server (via stdio)
server = MCPServerStdio(params={"command": "python", "args": ["server.py"]})
```

```

agent = Agent(mcp_servers=[server])

# Hosted service
agent = Agent(
    tools=[
        HostedMCPTool(
            tool_config={
                "type": "mcp",
                "server_label": "my-server",
                "server_url": "https://...",
                "require_approval": "never",
            }
        )
    ]
)

```

4.2.4 The Key Insight

MCP separates tool **definition** from tool **usage**. You can build servers that expose capabilities, and any MCP host can connect to discover and use those tools. This enables:

- **Reusability:** Write a tool once, use it from many applications
- **Ecosystem:** Use tools built by others, such as gitmcp.io
- **Dynamic discovery:** Hosts discover what tools exist at runtime
- **Process Isolation:** Tools run in separate processes from agents

4.3 Reflection

You've now learned how to build and use MCP servers. Take a moment to think about how this pattern could be useful.

Think about:

- **What other tools could you expose via MCP?** Database queries? API calls? File operations? Internal company services?
- **When would you use `@function_tool` vs. building an MCP server?** If the tool is specific to one agent and simple, use `@function_tool`. If you want to share tools across projects or need process isolation, use MCP.
- **How could you share your MCP servers with other developers?** MCP servers are just Python scripts. You could package them, containerize them, or host them as services.
- **What hosted MCP services would be useful for your work?** gitmcp.io is just one example. The ecosystem is growing with services for databases, APIs, and more.